ResearchOS: Research Compilation for Multi-Agent Scientific Discovery

Mengkun Liang Chenhao Xu Tianyu Zhu Guannan Liu Junjie Wu*

MIIT Key Laboratory of Data and Decision Intelligence
Beihang University, Beijing, China

{mengkun, ChenhXu, zhuty, liugn, wujj}@buaa.edu.cn

Abstract

Large language model agents increasingly support literature analysis, research ideation, experimentation, and scientific writing. Their collaboration depends on how source findings become research directions, how selected directions become experimental commitments, and how returned observations support manuscript claims. We introduce **ResearchOS**, an open-source multi-agent research system organized through *research compilation*, which gives these transitions explicit research artifacts and acceptance conditions. Source-linked reading and cross-paper synthesis establish a reusable research space. Within this space, a multi-agent evolutionary search develops structured study candidates using independent assessment, diagnosis-guided mutation, compatibility-aware crossover, and quality-diverse selection. Researcher-selected candidates undergo contribution review and study formalization before the approved plan is compiled into a contract for external execution. Returned records are checked against the study and connected to manuscript claims through evidence alignment and audit. An artifact-first runtime retains provenance, versions, researcher decisions, and recoverable state across the process. We present the architecture, evolutionary mechanism, execution interfaces, and representative system traces, illustrating how ResearchOS connects scientific exploration to executable investigations and evidence-supported reporting.

Keywords: multi-agent research • research compilation • evolutionary search • evidence provenance

Project website | GitHub repository | Laboratory

1 Introduction

Large language models (LLMs) increasingly participate in literature interpretation, hypothesis generation, code development, experimentation, and scientific writing. Earlier systems connected computational inference with laboratory experimentation [1–3]. More recent research agents coordinate several of these activities within a single workflow [4–7]. As their responsibilities expand, a central systems question is how separate contributions can accumulate into a coherent scientific investigation.

Consider two papers reporting different results for apparently similar methods. Their findings may differ because of evaluation conditions, implicit assumptions, or an unexamined mechanism. Reading and cross-paper synthesis expose these possibilities and motivate several research directions. One candidate may explain a boundary condition, while another may combine a proposed mechanism with an experiment that distinguishes competing explanations. Once the researcher selects a direction, its hypotheses, required comparisons, and evaluation criteria define the intended study. Experimental observations then determine the scope of the claims that can be reported. The underlying information

*Corresponding author: Junjie Wu (wujj@buaa.edu.cn).

evolves from documented evidence through proposed explanations to experimental commitments and findings.

For collaborating agents, this progression creates a specific computational problem. Source passages, developing research candidates, approved plans, execution records, and manuscript claims have different scientific meanings and different conditions for use. A hypothesis-generation agent can explore conjectural mechanisms using partial evidence. An executor receives a study whose required baselines and evaluation protocol have been approved. A writing agent must relate its conclusions to the observations actually returned. Across tools and interrupted sessions, these participants need access to the provenance, decision status, dependencies, and acceptance conditions of shared research objects.

We introduce **ResearchOS**, an open-source system that organizes these relationships through *research compilation*. Research compilation progressively transforms scientific information into structured artifacts with recorded sources, downstream roles, and checks governing their use. *Evidence compilation* builds a research space from source-linked reading and cross-paper synthesis. *Direction compilation* develops research candidates and formalizes the direction selected by the researcher. *Execution compilation* translates the approved plan into a validated handoff for external experimentation and admits returned records for scientific reporting. A persistent runtime maintains the artifacts, decisions, and recovery state connecting these transformations.

A central mechanism within this architecture is *evidence-grounded multi-agent evolutionary search*. The research space often supports several plausible explanations, each with a different mechanism or validation strategy. ResearchOS represents these alternatives through a structured *Idea Genome* and lets specialized agents propose, independently assess, mutate, and recombine candidates. Population-level selection considers scientific quality and diversity, retaining alternative research directions and the evidence from which they emerged. The resulting portfolio gives the researcher developed options for further scrutiny and study design.

The selected direction then passes through contribution review and research-plan formalization. Its approved hypotheses, baselines, and evaluation conditions become obligations in an executor-independent scientific contract. Returned experimental records are checked against that contract and aligned with the manuscript claims they can support. These handoffs preserve the decisions that make an investigation interpretable, while the runtime supports revision, human approval, and recovery across agents and tools.

At the architectural level, ResearchOS contributes research compilation, a model for carrying scientific commitments across evidence construction, research planning, execution, and reporting. At the algorithmic level, it contributes evidence-grounded multi-agent evolutionary search for developing and selecting structured research directions. We make these designs inspectable through the system's research objects, algorithm, execution interfaces, and representative traces from the open-source implementation [8]. Together, the two contributions connect scientific exploration with the execution and reporting of approved studies.

2 Related Work

Scientific automation has grown from specialized experimental systems into agents supporting broader research workflows. The Robot Scientist Adam connected hypothesis formation, experimental design, and laboratory records [1], while Coscientist demonstrated autonomous reasoning and tool use in chemistry [3]. The AI Scientist and The AI Scientist-v2 integrate idea generation, experimental search, and manuscript production [4, 5]. AI-Researcher, Agent Laboratory, and Denario develop additional approaches to research-agent coordination [6, 7, 9]. AutoResearchClaw and Claw AI Lab explore iterative implementation, researcher oversight, and inspectable execution environments [10, 11]. Arbor maintains a hypothesis tree that links research directions, experimental outcomes, and accumulated insights [12]. PaperBench studies agents' ability to reproduce specified research artifacts [13]. Together, these systems

show how automation can extend across scientific tasks and highlight the importance of carrying research context from one activity to the next.

System	Research exploration	Persistent research state	Execution and reporting emphasis
AI Scientist-v2 [5]	Agentic tree search over experiments	Evolving experiment plans and code	Autonomous experiments and paper generation
AI-Researcher [6]	Agent-supported idea development	Literature notes and research plans	Multi-agent implementation and manuscript preparation
Co-Scientist [14]	Hypothesis debate, ranking, and evolution	Evaluated hypothesis portfolio	Scientist-facing hypotheses and proposals
HypoEvolve [15]	Genetic search over hypotheses	Candidate population and lineage	Hypothesis assessment and selection
Arbor [12]	Hypothesis-tree refinement	Hypotheses, artifacts, evidence, and insights	Isolated experiments and held-out improvement gate
AutoResearchClaw [10]	Multi-agent debate and iterative refinement	Cross-run research experience	Self-healing execution and verifiable reports
ResearchOS	Source-linked, evidence-aware genome evolution	Versioned evidence, candidates, and approved study	Contract-based external execution and evidence-to-claim audit

Table 1. Architectural emphasis of representative research-agent systems, based on their published designs. The comparison describes the principal scientific objects and handoffs of each system.

Scientific literature provides both the material for research exploration and the basis for subsequent claims. OpenScholar investigates citation-grounded synthesis over large scientific corpora [16], and ResearchAgent uses literature and feedback to develop research proposals [17]. Attribution and scientific claim verification research examines the relationship between generated statements and identifiable supporting evidence [18–20]. The FAIR principles and the PROV model establish influential practices for reusable research data and provenance relationships [21, 22]. These perspectives motivate retaining source identity, reading coverage, and evidence-use conditions as research material moves into candidate generation and review.

Research ideation also lends itself to iterative search. ResearchAgent incorporates reviewing agents, while evaluations of LLM-generated proposals distinguish scientific novelty from feasibility [17, 23]. Co-Scientist uses agentic generation, critique, and refinement to develop hypotheses [14]; HypoEvolve applies genetic search, and EvoDiverse considers diversity among evolving scientific alternatives [15, 24]. EvoNarrator contributes structured patterns for scientific hypothesis generation [25]. In computational search, FunSearch and ReEvo demonstrate how language-model reasoning can guide evolutionary improvement [26, 27]. Multiobjective and quality-diversity methods offer corresponding principles for retaining complementary candidates [28, 29]. ResearchOS combines these ideas around structured study directions whose mechanisms, candidate contributions, evidence roles, and validation plans remain inspectable across generations.

Research workflow and reproducibility studies provide another foundation. Pegasus formalizes execution dependencies and recovery [30]. Reproducibility guidance emphasizes preserving data, code, configuration, and experimental conditions [31, 32], while preregistration clarifies the role of decisions established before results are observed [33]. Work on auditable research agents and multi-agent coordination likewise examines inspectable actions and responsibilities [34, 35]. ResearchOS incorporates these considerations into the transition from a developing research direction to an approved study and, subsequently, to admitted experimental evidence.

Table 1 situates ResearchOS among representative systems according to the scientific objects they explore, the research state they preserve, and the execution or reporting interfaces they emphasize. These dimensions clarify its design focus on source-linked research directions and explicit handoffs from approved plans to experimental evidence and manuscript claims.

3 ResearchOS Architecture

Figure 1 presents the system as three connected research compilations supported by a shared runtime. Evidence compilation organizes literature and project materials into a source-linked research space. Direction compilation develops research candidates through multi-agent evolution and formalizes the study approved by the researcher. Execution compilation produces project-specific skills and a scientific contract for external tools, then checks the records returned under that contract. The evidence-to-manuscript workflow uses admitted results together with the approved study and literature evidence to develop a reviewable scientific account. The architecture gives each stage a defined research object to produce and a clear basis for downstream use.

A research candidate and an approved study carry different scientific obligations. Candidates preserve proposed mechanisms, supporting material, and alternative interpretations for further exploration. Once the researcher approves a study, its blueprint establishes hypotheses, required baselines, and evaluation conditions for execution. Returned records are checked against the corresponding contract before contributing to scientific reporting. The runtime preserves these objects and review decisions across sessions, so new results can also inform renewed reading and exploration. Manuscript checks focus on the connection between reported claims, admitted observations, and their conditions of interpretation.

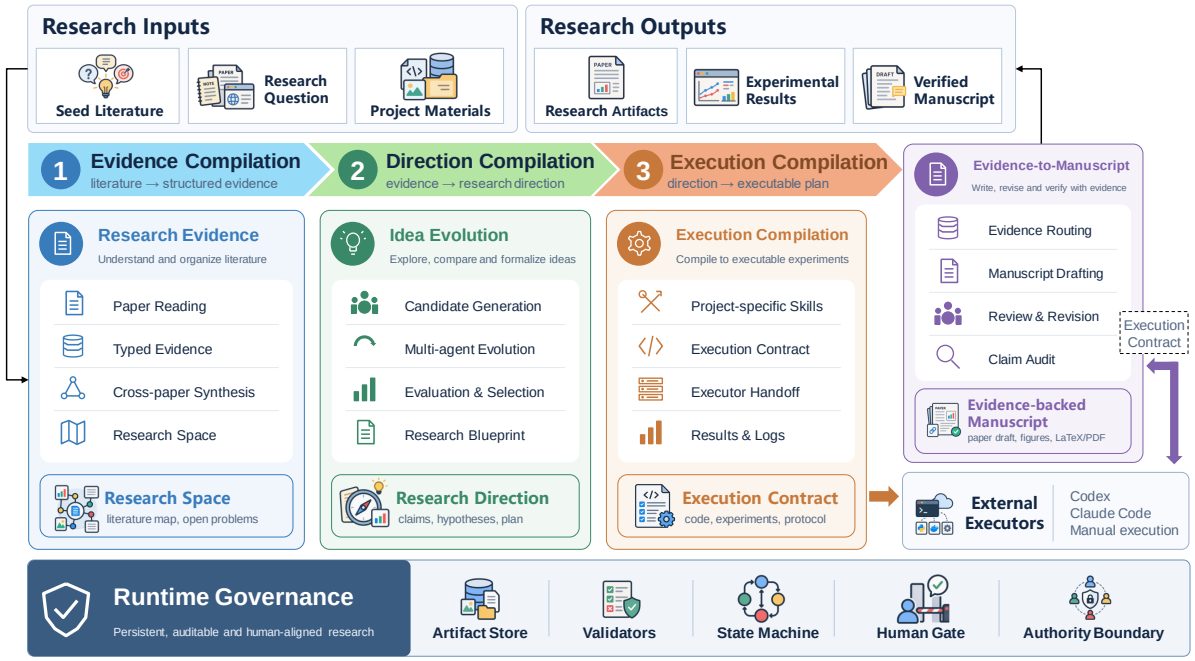


Figure 1. ResearchOS architecture. Three research compilations connect source-linked evidence, evolutionary research directions, and approved execution contracts. The evidence-to-manuscript workflow develops research outputs from admitted evidence, while the persistent runtime supplies artifact storage, validation, state management, human approval, and authority boundaries.

3.1 Research Objects and Compilation Boundaries

ResearchOS represents scientific content as persistent, versioned *artifacts*, each identifying a research object and the sources, dependencies, and decisions associated with it. A reading note records inspected material, while a *research space* organizes source-linked findings, comparisons, and open problems. A *research candidate* specifies a developing direction whose mechanism and validation design may evolve. An *accepted research plan* records the hypotheses, comparisons, and evaluation conditions approved for a study. An *execution contract* conveys those requirements to external tools. Finally, a *manuscript claim* connects a scientific statement to relevant literature or admitted experimental observations. These object types make the scientific status and intended use of shared information explicit for downstream agents.

Research compilation governs how these objects acquire new downstream roles. Evidence compilation establishes the documented basis for exploration. Direction compilation evaluates evolving candidates and produces an approved research blueprint after contribution review and formalization. Execution compilation derives a checked handoff from this plan and receives the resulting experimental records. Generative reasoning supports literature interpretation and research-direction development. The accepted plan is then converted into its execution contract by deterministic compilation and validation. Together, these mechanisms allow exploratory proposals to become explicit scientific obligations while preserving their provenance and intended tests. Table 2 identifies the main handoffs and their acceptance conditions; manuscript development uses their admitted outputs as the basis for evidence-supported reporting.

Transition	Scientific handoff	Acceptance condition
Evidence compilation	Literature and project materials → source-linked research space	Source identity, reading coverage, and evidence-use conditions
Direction compilation	Research space and candidates → accepted study	Candidate assessment, contribution review, and researcher approval
Execution compilation	Accepted study → contract and admitted records	Plan identity, required comparisons, and return-package validation
Manuscript development	Accepted evidence → research claims and draft	Claim-to-evidence alignment and reporting-scope review

Table 2. ResearchOS scientific handoffs. Manuscript development consumes admitted outputs of execution compilation and is shown as their downstream use.

Each handoff retains the decisions needed to interpret later work. Literature notes record reading coverage, candidate versions preserve their lineage and evidence, and external returns identify the exact study plan and contract they address. When the researcher revisits a direction or changes an experimental protocol, these dependencies reveal which downstream artifacts require renewed review.

3.2 Shared Runtime and Decision Authority

The shared runtime brings together an artifact store, validators, a state machine, human approval gates, and authority boundaries, as shown in Figure 1. The artifact store maintains versions and dependencies; validators check schemas, required files, source and study identities, content fingerprints, and declared completion conditions. A content fingerprint identifies the version of an object on which downstream work depends. The state machine coordinates progression and recovery. After a researcher revises an approved study, these records identify the dependent artifacts that require revalidation while preserving upstream research material.

The runtime also assigns explicit decision responsibilities. Language-model agents interpret sources, develop and assess research directions, and draft scientific explanations. A deterministic controller manages candidate identities, evolutionary scheduling, population selection, and interface validation.

External executors implement experiments under the approved contract. At *human gates*, the researcher selects directions, approves study plans, and authorizes changes affecting scientific commitments. These decisions remain linked to the relevant artifacts, enabling implementation repairs to proceed within the contract and protocol changes to return for research review.

The persistent workspace supports task resumption and renewed scientific decisions. Resuming a paused task uses the recorded state and validated artifacts. Re-entering an earlier decision retains its history and triggers appropriate checks on dependent work. Different agents and tools can therefore contribute across sessions while relying on the same research record. The next sections follow the development of this record from evidence construction through research-direction evolution and external execution.

4 Evidence Compilation

Research opportunities frequently arise from relationships among existing findings. A disagreement may depend on different measurements or assumptions; a method developed for one setting may expose a useful mechanism in another. Identifying such connections requires comparable accounts of previous studies, together with a record of the evidence supporting each interpretation. Evidence compilation develops these accounts into a research space that agents can revisit during ideation, contribution review, and scientific writing.

4.1 Source-Linked Reading and Evidence Use

Literature discovery draws on retrieval, researcher-provided papers, citation relationships, and neighboring work. ResearchOS checks bibliographic identity, manages duplicates, and maintains reading queues for the investigation. For each paper, the workspace records both document availability and the material actually inspected. Reading coverage distinguishes full-text examination, specified sections or page ranges, and abstract-level inspection. A later agent can therefore locate the evidence already reviewed and identify the material that still requires reading.

The reader turns inspected sources into structured notes covering research questions, mechanisms, methods, experimental designs, results, and conditions of applicability. It retains page, section, figure, or table anchors where available, alongside accounts of the authors' design rationale and claimed contributions. The notes distinguish reported observations from their proposed explanations and theoretical arguments. These distinctions matter when a measured difference between two studies becomes the starting point for a new mechanistic hypothesis.

ResearchOS records an *evidence permission* for the downstream use of source-linked material. The permission reflects what has been inspected and the support available for a particular statement. An abstract reporting improved performance may justify investigating the method; a mechanistic explanation of that improvement calls for the relevant experimental details and conditions. Evidence permissions thus allow partial readings to contribute to discovery while retaining the additional checks required when a statement is used to support a stronger claim. They accompany notes and proposed cross-paper relations into subsequent research stages.

Agents can also introduce hypotheses through analogy, model knowledge, or a possible connection between research areas. Such proposals enter the research space as conjectures, with their motivating sources and open evidence requirements recorded. This supports exploration without prematurely elevating an interpretation into an established finding. The unresolved requirements can guide subsequent reading and become explicit considerations when a research direction is assessed or formalized.

4.2 Cross-Paper Synthesis and the Research Space

Cross-paper synthesis compares studies by problem, mechanism, method, evaluation condition, and contribution. Bibliographic citation links provide documented relationships; model-assisted synthesis identifies candidate mechanism families, differences in outcomes, boundary conditions, and possible transfers between domains. Each synthesized connection is associated with the underlying source notes. ResearchOS uses these relationships to construct a *research space* that combines a domain-level map, comparative literature materials, and opportunities for further investigation.

For the two studies introduced earlier, this synthesis might isolate an evaluation condition under which their results diverge. That observation can motivate competing directions. One candidate might seek an explanation of the boundary condition, another might adapt a mechanism to the more difficult setting, and a third might design a test that separates rival explanations. The research space preserves the observed difference and its source, while each candidate supplies its own proposed interpretation. Consequently, agents can compare potential directions against a shared account of the literature and the scientific questions it leaves open.

These knowledge assets also support activities beyond the immediate search. An optional survey workflow can assemble the notes into a taxonomy and cross-study synthesis; later tasks can draw on the same comparisons when checking related work, specifying hypotheses, or writing a manuscript. Evidence compilation therefore gives each new research direction a reusable basis in documented knowledge while keeping its novel propositions available for subsequent examination.

5 Direction Compilation through Multi-Agent Evolution

The research space may support several plausible mechanisms and experimental approaches. Exploring them calls for a search procedure that can refine promising proposals and retain alternatives with different scientific strengths. ResearchOS represents this exploration as a population of research directions. Language-model agents generate, review, and evolve their scientific content, while a deterministic controller manages candidate identity, lineage, and population updates. The search produces a portfolio for researcher selection, followed by the contribution review and formalization required to establish an executable study.

5.1 Scientific Representation and Population Initialization

Evolutionary operations require a representation that exposes the substance of a proposed study. ResearchOS uses an *Idea Genome* with eleven scientific fields: the research problem, opportunity, challenged assumption, central thesis, mechanism, design or artifact, contribution, provisional hypotheses, validation logic, boundary conditions, and risks. Each *gene* denotes one of these fields and carries relevant evidence references and recorded revisions. A candidate dossier joins the genome to its assessments and development history. Stable identifiers, versions, generations, and parent relationships preserve the candidate's evolutionary lineage.

The Idea Genome makes scientific revisions inspectable at the level of their intended contribution. An evaluator may identify a coherent mechanism whose proposed experiments cannot distinguish it from a competing explanation, or a testable hypothesis whose contribution remains underspecified. Mutation can then target the relevant fields, while crossover can combine compatible mechanisms and validation designs with their provenance intact. Because the representation links explanation, contribution, and test, reviewers can assess what a revision changes in the proposed investigation.

Genes retain the evidential role of the material they use. A candidate may build on a documented finding, draw inspiration from an adjacent result, or propose a conjectural mechanism. If two experiments differ under a known condition, for example, the observed difference can motivate a new explanation while the genome's validation fields specify measurements that would distinguish that explanation from

alternatives. This linkage lets the system assess whether a proposed investigation is well conceived, and later lets experimental evidence address whether its hypothesis is supported.

Population initialization uses complementary discovery routes, including open questions, conflicting results, challenged assumptions, mechanism transfer, and potentially compatible methods. Project goals, source evidence, and available resources shape the initial seeds. Agents enrich these seeds into complete candidate dossiers, after which assessment occurs in review contexts separate from proposal generation. The evolution role then plans targeted revisions and recombinations. The same LLM backend may serve several roles; their distinct inputs, responsibilities, and review boundaries define the collaboration, while the controller records the search state and its history.

5.2 Diagnosis-Guided Variation and Population Selection

Candidate assessments identify where an investigation could be strengthened and provide the quality profile used for population selection. The implementation assigns three core scores, each in the interval $[1, 5]$, to an assessed candidate c. The resulting quality vector is

$$\mathbf{f}(c) = \big(f_{\text{value}}(c),\ f_{\text{mechanism}}(c),\ f_{\text{contribution}}(c)\big), \tag{1}$$

Its components assess *research value*, *mechanism integrity*, and *contribution distinctiveness*. Evidence calibration and validation feasibility are recorded separately, together with publication-profile fit and scientific upside. The scores compare proposed research designs; the diagnostics indicate what evidence, resources, or experimental detail still need attention. These assessments guide candidate comparison while preserving a separate record of the scientific and practical questions still requiring attention.

Assessment is separated from generation through designated agent roles and review contexts. Evaluators inspect candidate dossiers without the preceding generation dialogue, identify strengths, and diagnose the most consequential weakness in the research design. This diagnosis guides subsequent variation. Because model-based judgments depend on the evaluation context [36], admitted offspring and their parents are reviewed under common criteria. The system retains the scores, diagnoses, modified fields, and parentage, making the basis for later researcher decisions inspectable.

Mutation revises an individual candidate according to a diagnosed bottleneck. Its plan specifies the gene to improve and the scientific commitments to preserve. Consider a candidate prompted by the disagreement between two studies. An initial proposal may simply expect a new method to perform better. A targeted mutation can develop a mechanistic explanation tied to the differing study conditions and add a baseline or ablation that would discriminate it from another explanation. Its recorded gene changes identify the revised mechanism and the evidence needed to evaluate it.

Crossover searches for complementarity between candidates. It begins with a scientific compatibility check across their problems, mechanisms, and proposed validations. A gene-donor map then identifies which parent contributes each scientific component, and the resulting child explains their joint rationale. In the same illustrative setting, an explanatory mechanism from one candidate may be combined with a discriminating experimental design from another, provided that the design actually tests the combined claim. The donor record preserves the scientific origin of the new direction, while compatibility review checks that its assumptions and proposed tests remain coherent.

Offspring pass schema, provenance, and gene-change checks before joining the comparison population. The controller records modified fields and identifies cosmetic changes, deviations from protected scientific content, and added complexity whose scientific purpose has not been established. Independent scientific assessment then evaluates the admissible revisions. Parents remain in the comparison, so exploration can preserve established alternatives while incorporating genuinely developed proposals. If an evolution attempt yields no admitted offspring, the prior population remains available with its earlier assessments.

Let P_g denote the active population at generation g, and let C_g denote the offspring admitted for comparison. The generation update operates on $P_g \cup C_g$. Among scored, structurally valid candidates, a dominates b when it is no worse on every scientific-quality objective and better on at least one,

$$a \succ b \quad \Longleftrightarrow \quad \begin{aligned} & f_j(a) \geq f_j(b) \quad \forall j \in \{1,2,3\}, \\ & f_j(a) > f_j(b) \quad \text{for some } j. \end{aligned} \tag{2}$$

Repeatedly identifying nondominated candidates creates Pareto layers, following multiobjective evolutionary selection [28]. The controller uses these layers together with coverage of candidate families and structural diversity. A family groups proposals with similar problem, mechanism, and contribution fields, using a lightweight field-level token-overlap rule. The resulting selection retains scientifically promising candidates across distinct explanatory directions. Appendix A details the ordering, family limits, and tie-breaking conditions used by the implementation.

ResearchOS separates *population survival* from *portfolio presentation*. Survival determines which candidates continue into later generations. Portfolio presentation selects a smaller set of directions that exposes different mechanisms and research trade-offs to the researcher, including a marked high-upside alternative when appropriate. The researcher can inspect the lineage and evidence of these choices, continue evolving the population, focus on one direction, or reconsider an earlier selection without losing the alternatives already developed.

Algorithm 1 summarizes the search. The standard workflow constructs the initial population and performs one generation update before presenting a portfolio. The researcher can authorize additional generations, with their outputs and resource use recorded. Retained populations and lineages provide a documented set of alternatives for subsequent exploration and research-plan approval.

Algorithm 1 Research-direction evolution

Require: Research space R, constraints U, authorized generations $G \geq 1$
Ensure: Candidate portfolio and retained lineage
Generate seeds from R, U and enrich into P_0
Validate and independently assess P_0
Save candidate provenance and assessments
$g \leftarrow 0$
while $g < G$ **do**
    Select parents by quality and family coverage
    Diagnose bottlenecks and plan mutations
    Check compatibility and plan crossovers
    Generate and validate offspring C_g
    if $C_g \neq \emptyset$ **then**
        Reassess $P_g \cup C_g$ together
    end if
    Update P_{g+1} by Pareto and family rules
    Save survivors, alternatives, and lineage
    $g \leftarrow g + 1$
end while
Present a diverse researcher-facing portfolio

5.3 From Selected Candidates to Accepted Research Plans

An evolved candidate presents a research opportunity with a proposed mechanism and validation strategy. Before it guides experimentation, the researcher needs a defined account of the claims under investigation, the alternatives to compare, and the observations that would distinguish them. ResearchOS therefore connects evolutionary search to execution through contribution review and study formalization. This step develops a selected direction into an investigation with explicit scientific commitments.

Contribution review examines two complementary questions. A *collision assessment* compares the candidate's mechanism and design with nearby studies to identify overlapping explanations or

implementations. A *contribution-distance assessment* asks what additional understanding or capability the proposed investigation could provide if its claims were supported. Both draw on source-checked descriptions of related mechanisms and their design rationale. Together, these assessments clarify how the proposed mechanism relates to prior work and what an experiment could establish about its contribution [37].

Formalization produces a research blueprint, a registry of prospective scientific claims, hypotheses, and an experimental plan. The blueprint describes the proposed mechanism and design, and the claim registry states the propositions under examination. The hypotheses and plan specify datasets, required baselines, metrics, resource conditions, and comparisons. Cross-references among these artifacts show what evidence each proposition requires and which outcomes would count against it. The implementation calls the corresponding disconfirming or stopping conditions *kill criteria*. In the running example, the formalized study would identify the condition under which results diverge and predefine comparisons capable of distinguishing the candidate explanations.

An independent review checks the coherence of the blueprint, claims, hypotheses, and evaluation protocol before the researcher approves the study. Candidate identifiers and content fingerprints bind the reviewed plan to the direction from which it developed. Other candidates remain available for separate development. The resulting accepted plan establishes the scientific obligations that the next stage must preserve and supplies the reference against which external execution will be checked.

6 Execution Compilation and Scientific Reporting

A reviewed research plan specifies what will be tested. Implementation requires a different set of decisions about code, dependencies, computational resources, and execution tools. ResearchOS connects them through execution compilation, which conveys the accepted scientific requirements to an external executor and checks the returned records against those requirements. The admitted evidence then becomes the basis for manuscript development. This arrangement allows implementation work to proceed with clear scientific obligations and a durable account of what was actually performed.

6.1 Compiling Scientific Commitments for External Execution

The `research-reboost` compiler deterministically transforms the accepted research package into a validated execution contract. It preserves the central hypotheses, mechanism constraints, required baselines, evaluation protocol, resource conditions, claim boundaries, and expected outputs. Input identities and content fingerprints tie the contract to the approved plan. As a result, an external coding agent can work from a concrete specification of the study. Repairing a software dependency falls within implementation, whereas removing a required baseline changes the planned scientific comparison and requires renewed research approval.

The contract is then specialized into a project-specific suite of *skills*. A skill is a reusable task specification containing domain guidance, permitted operations, relevant inputs, expected outputs, and completion conditions. Model-based assistance adapts the skill instructions to the proposed methods and available resources; deterministic checks verify the generated files against the authoritative contract. Executor guidance, allowed paths, and output schemas make the handoff operational without delegating the researcher's scientific decisions to the execution tool.

The interface supports external coding agents, interactive executor sessions, and researcher-run experiments. Documented integrations include Codex CLI and Claude Code. A separate resource-preparation gate enables the researcher to review datasets, reference implementations, model assets, and licenses before authorizing experiments. An executor can then iterate through implementation, execution, diagnosis, and repair within the granted scope. Proposed changes to hypotheses, required

comparisons, or evaluation conditions return to the research-decision process with the original plan available for inspection.

ResearchOS retains the approved scientific plan in its own workspace and supplies different execution tools with a common contract and return format. The executor supplies code, configurations, measurements, and logs; the research runtime retains the plan against which those products are interpreted. The same approved study can thus be handed to another supported tool while maintaining a stable description of its scientific objectives and expected evidence.

6.2 Admitting Experimental Evidence

An external execution return contains a research report, structured result pack, execution-status record, run manifest, writing-handoff facts, and validation receipt. The manifest relates reported outcomes to source observations, configurations, logs, and generated tables or figures. Before admitting these records into manuscript development, ResearchOS checks them against the current study identity and execution contract. The checks cover declared status, required files, evidence references, permitted paths, content hashes, and consistency between the manifest and the files it identifies. Appendix B describes the interface and its derived writing artifacts.

The admission procedure records the work actually performed, including any explicitly partial execution. Resource-preparation and protocol dry runs carry their own status, so the workspace preserves which actions and observations are available for scientific interpretation. A failed interface check produces a typed rejection identifying missing or inconsistent material and permits a corrected return. These checks establish the identity, provenance, and contract conformity of the returned records. Their scientific interpretation draws on the accepted evaluation design, observed results, and hypotheses under examination.

After admission, the runtime constructs an experimental evidence package and a result-to-claim mapping. The mapping relates observations to the prospective hypotheses, baselines, and measurements defined before execution. Partial observations may justify a narrower statement, and a finding that challenges the original explanation can motivate its revision. Subsequent writing agents can therefore trace reported observations to the approved study and its actual execution.

6.3 Developing a Manuscript from Accepted Evidence

Manuscript development draws on three connected resources. The accepted plan defines the research question, proposed mechanism, and intended tests. Admitted execution records provide the experimental observations. The source-linked research space supplies the prior findings and context needed to interpret them. ResearchOS maintains an alignment matrix relating proposed contributions, hypotheses, design choices, experiments, and analyses. This structure lets the writer explain why each comparison was conducted and locate the evidence for the conclusions it supports.

A shared writing plan organizes the scientific argument before individual sections are drafted. The method establishes the design and its rationale, the experiments describe the measurements, and the analysis interprets them in relation to the hypotheses and previous research. The introduction and abstract are developed from that established argument. A shared *storyline* records the central research problem, mechanism, contribution, and evidence-supported findings across writing sessions. Section-level agents can therefore work with a consistent account of the study while the researcher revises the narrative and its supporting material.

The evidence-to-manuscript workflow shown in Figure 1 combines evidence routing, manuscript drafting, review and revision, and a final *claim audit*. The audit checks manuscript statements against source attributions, admitted results, and the conditions under which observations were obtained. It compares reported quantities with recorded outputs and examines the scope of their interpretation. For example, a measured result from one experimental condition should remain connected to that condition

when it appears in the analysis or conclusion. Statements with unresolved evidence requirements are surfaced for researcher review. The process produces revisable LaTeX sources and a compiled PDF while preserving the underlying research objects.

The accepted plan establishes the hypotheses and tests before experimentation. Admitted observations then provide the basis for interpreting outcomes and reporting claims. ResearchOS preserves this relationship within one research record, so the manuscript remains connected to the scientific commitments, execution conditions, and evidence on which its arguments depend.

7 Implementation and System Demonstration

ResearchOS is implemented in Python with a command-line interface, persistent workspaces, specialized agents, validators, and a state machine. Model backends and external executors use interfaces defined by project artifacts and the approved execution contract. The repository documents the implementation [8]. Figure 2 presents representative interfaces for literature review, idea development, executor handoff, and manuscript writing. Appendix C maps the implementation's task identifiers to the architecture.

A workspace retains the state of an investigation across commands and sessions. Listing 1 shows project initialization, execution, status inspection, and resumption after installation and environment configuration. The shell variable `WS` abbreviates the workspace path. Each command accesses the same stored research record, so interrupted work and earlier scientific decisions remain available when the researcher returns.

```
WS=./workspace/project-a
python -m researchos.cli configure-llm
python -m researchos.cli doctor --workspace "$WS"
python -m researchos.cli init-workspace \
  --workspace "$WS" --project-id project-a \
  --topic "A research question"

python -m researchos.cli run --workspace "$WS"
python -m researchos.cli status --workspace "$WS"
python -m researchos.cli resume --workspace "$WS"
```

Listing 1. Initializing, running, and resuming a persistent research workspace.

Figure 2(a) shows literature-reading progress, processed page ranges, and source-linked notes. Its inset reveals the current reading task and the materials retained for cross-paper synthesis. Panel (b) presents a research-direction candidate involving semantic identifiers in conditional treatment-effect modeling. The dossier brings together its research question, mechanism, contribution, provisional hypotheses, and evidence requirements, supporting comparison and researcher selection.

Panel (c) shows the human gate for external resource preparation. The current hypothesis, required baselines, and executor options accompany a bounded execution contract, allowing the researcher to authorize the handoff while retaining the study's scientific commitments. Panel (d) presents a compiled literature-review draft on LLMs in causal inference, developed through the optional survey workflow from accumulated reading and synthesis artifacts.

8 Conclusion

ResearchOS contributes an evidence-grounded evolutionary search for developing scientific directions and a research compilation architecture that carries selected directions into executable, inspectable

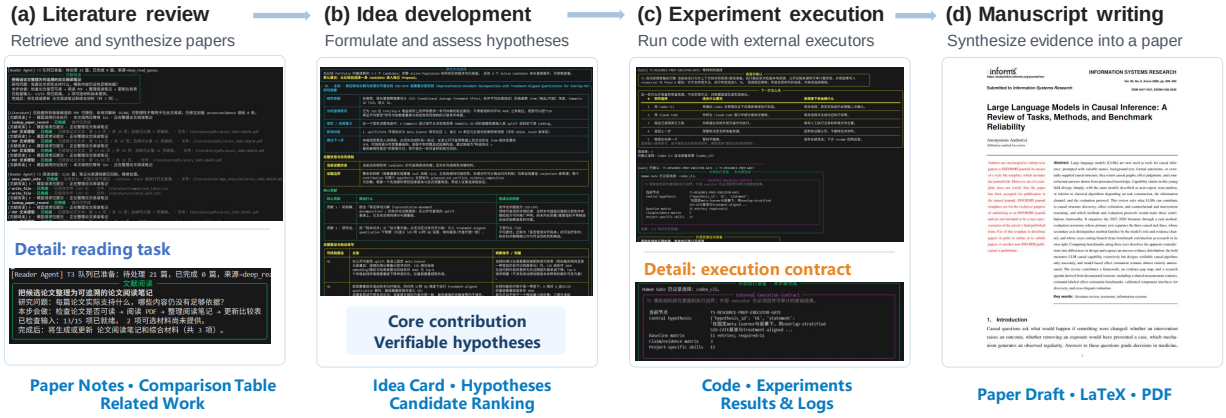


Figure 2. Representative ResearchOS interfaces from distinct research tasks. (a) Literature review records reading progress and source-linked notes. (b) Idea development presents a candidate's research question, proposed contribution, and verifiable hypotheses. (c) Experiment execution is illustrated by the researcher approval gate and contract used to hand a study to external tools. (d) Manuscript writing is illustrated by a compiled LaTeX/PDF review draft from the literature-survey workflow.

studies. Source-linked reading makes prior findings reusable during search, while the Idea Genome, diagnosis-guided variation, and quality-diverse selection retain the scientific structure of alternative proposals. Once a direction is accepted, execution contracts preserve its hypotheses and evaluation requirements across external tools, and evidence admission links the returned observations to manuscript claims. Persistent artifacts, validation, and researcher approval connect both designs throughout the investigation. By connecting evidence-grounded direction evolution with explicit research compilation, ResearchOS provides an open-source architecture for developing research ideas into executable studies and evidence-supported scientific reports.

References

[1] Ross D. King, Jem Rowland, Stephen G. Oliver, Michael Young, Wayne Aubrey, Emma Byrne, Maria Liakata, Magdalena Markham, Pinar Pir, Larisa N. Soldatova, Andrew Sparkes, Kenneth E. Whelan, and Amanda Clare. The automation of science. *Science*, 324(5923):85–89, 2009. doi: 10.1126/science.1165620. URL https://www.science.org/doi/10.1126/science.1165620.

[2] Hanchen Wang, Tianfan Fu, Yuanqi Du, Wei Gao, et al. Scientific discovery in the age of artificial intelligence. *Nature*, 620:47–60, 2023. doi: 10.1038/s41586-023-06221-2. URL https://www.nature.com/articles/s41586-023-06221-2.

[3] Daniil A. Boiko, Robert MacKnight, Ben Kline, and Gabe Gomes. Autonomous chemical research with large language models. *Nature*, 624:570–578, 2023. doi: 10.1038/s41586-023-06792-0. URL https://www.nature.com/articles/s41586-023-06792-0.

[4] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist: Towards fully automated open-ended scientific discovery, 2024. URL https://arxiv.org/abs/2408.06292. arXiv:2408.06292.

[5] Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist-v2: Workshop-level automated scientific discovery via agentic tree search, 2025. URL https://arxiv.org/abs/2504.08066. arXiv:2504.08066.

[6] Jiabin Tang, Lianghao Xia, Zhonghang Li, and Chao Huang. AI-researcher: Autonomous scientific innovation. In *Advances in Neural Information Processing Systems*, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/file/0d904d300a105809a2114d727851e759-Paper-Conference.pdf.

[7] Samuel Schmidgall, Yusheng Su, Ze Wang, Ximeng Sun, Jialian Wu, Xiaodong Yu, Jiang Liu, Michael Moor, Zicheng Liu, and Emad Barsoum. Agent laboratory: Using LLM agents as research assistants. In *Findings of the Association*

for Computational Linguistics: EMNLP 2025, pages 5977–6043. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.findings-emnlp.320. URL https://aclanthology.org/2025.findings-emnlp.320/.

[8] DIG-ResearchOS. DIG-ResearchOS: Source code and technical documentation. GitHub repository, 2026. URL https://github.com/MengkunLiang/DIG-ResearchOS. Public main branch, accessed 18 October 2026.

[9] Francisco Villaescusa-Navarro, Boris Bolliet, Pablo Villanueva-Domingo, Adrian E. Bayer, et al. Denario project: Deep knowledge artificial intelligence agents for scientific discovery. *PRX Intelligence*, 1:021001, 2026. doi: 10.1103/kk55-gc95. URL https://journals.aps.org/prxintelligence/abstract/10.1103/kk55-gc95.

[10] Jiaqi Liu, Shi Qiu, Mairui Li, Bingzhou Li, Haonian Ji, et al. AutoResearchClaw: Self-reinforcing autonomous research with human–AI collaboration, 2026. URL https://arxiv.org/abs/2605.20025. arXiv:2605.20025.

[11] Fan Wu, Cheng Chen, Zhenshan Tan, Taiyu Zhang, Xinzhen Xu, et al. Claw AI lab: An autonomous multi-agent research team, 2026. URL https://arxiv.org/abs/2605.22662. arXiv:2605.22662.

[12] Jiajie Jin, Yuyang Hu, Kai Qiu, Qi Dai, Chong Luo, Guanting Dong, Xiaoxi Li, Tong Zhao, Xiaolong Ma, Gongrui Zhang, Zhirong Wu, Bei Liu, Zhengyuan Yang, Linjie Li, Lijuan Wang, Hongjin Qian, Yutao Zhu, and Zhicheng Dou. Toward generalist autonomous research via hypothesis-tree refinement, 2026. URL https://arxiv.org/abs/2606.11926. arXiv:2606.11926.

[13] Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, Leon Maksin, Rachel Dias, Evan Mays, Benjamin Kinsella, Wyatt Thompson, Johannes Heidecke, Amelia Glaese, and Tejal Patwardhan. PaperBench: Evaluating AI's ability to replicate AI research. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 56843–56873. PMLR, 2025. URL https://proceedings.mlr.press/v267/starace25a.html.

[14] Juraj Gottweis, Wei-Hung Weng, Alexander Daryin, Tao Tu, et al. Accelerating scientific discovery with Co-Scientist. *Nature*, 655:487–496, 2026. doi: 10.1038/s41586-026-10644-y. URL https://www.nature.com/articles/s41586-026-10644-y.

[15] Jieyuan Liu, Mengzhou Hu, Jefferson Chen, JungHo Kong, Pratibha Jagannatha, Yiming Gao, Dexter Pratt, Hsin-Yuan Lee, Zhiting Hu, Trey Ideker, Wei Wang, Eric P. Xing, and Zhen Wang. HypoEvolve: Genetic algorithms enable multi-agent LLMs to discover scientific hypotheses, 2026. URL https://arxiv.org/abs/2609.15938. arXiv:2609.15938.

[16] Akari Asai, Jacqueline He, Rulin Shao, Weijia Shi, et al. Synthesizing scientific literature with retrieval-augmented language models. *Nature*, 650:857–863, 2026. doi: 10.1038/s41586-025-10072-4. URL https://www.nature.com/articles/s41586-025-10072-4.

[17] Jinheon Baek, Sujay Kumar Jauhar, Silviu Cucerzan, and Sung Ju Hwang. ResearchAgent: Iterative research idea generation over scientific literature with large language models. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 6709–6738. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.naacl-long.342. URL https://aclanthology.org/2025.naacl-long.342/.

[18] Hannah Rashkin, Vitaly Nikolaev, Matthew Lamm, Lora Aroyo, Michael Collins, Dipanjan Das, Slav Petrov, Gaurav Singh Tomar, Iulia Turc, and David Reitter. Measuring attribution in natural language generation models. *Computational Linguistics*, 49(4):777–840, 2023. doi: 10.1162/coli_a_00486. URL https://aclanthology.org/2023.cl-4.2/.

[19] Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. Enabling large language models to generate text with citations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 6465–6488. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.emnlp-main.398. URL https://aclanthology.org/2023.emnlp-main.398/.

[20] David Wadden, Shanchuan Lin, Kyle Lo, Lucy Lu Wang, Madeleine van Zuylen, Arman Cohan, and Hannaneh Hajishirzi. Fact or fiction: Verifying scientific claims. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 7534–7550. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.emnlp-main.609. URL https://aclanthology.org/2020.emnlp-main.609/.

[21] Mark D. Wilkinson, Michel Dumontier, I. Jsbrand Jan Aalbersberg, et al. The FAIR guiding principles for scientific data management and stewardship. *Scientific Data*, 3:160018, 2016. doi: 10.1038/sdata.2016.18. URL https://doi.org/10.1038/sdata.2016.18.

[22] Luc Moreau and Paolo Missier. PROV-DM: The PROV data model. W3C Recommendation, 2013. URL https://www.w3.org/TR/prov-dm/.

[23] Chenglei Si, Diyi Yang, and Tatsunori Hashimoto. Can LLMs generate novel research ideas? a large-scale human study with 100+ NLP researchers. In *International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/ea94957d81b1c1caf87ef5319fa6b467-Abstract-Conference.html.

[24] Haorui Wang, Parshin Shojaee, Kazem Meidani, Kunyang Sun, José Miguel Hernández-Lobato, Teresa Head-Gordon, Jiajun He, Chandan K. Reddy, Chao Zhang, and Yuanqi Du. Towards diverse scientific hypothesis search with large language models. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 129851–129891. PMLR, 2026. URL https://proceedings.mlr.press/v306/wang26gx.html.

[25] Xiaoying Le, Pengfei Qian, Yuanzhao Zhai, Xu Zhang, Qian Liu, Dawei Feng, and Bo Ding. EvoNarrator: Modeling scientific evolution for feasible hypothesis generation. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11846–11865. Association for Computational Linguistics, 2026. doi: 10.18653/v1/2026.acl-long.544. URL https://aclanthology.org/2026.acl-long.544/.

[26] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, et al. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024. doi: 10.1038/s41586-023-06924-6. URL https://www.nature.com/articles/s41586-023-06924-6.

[27] Haoran Ye, Jiarui Wang, Zhiguang Cao, Federico Berto, Chuanbo Hua, Haeyeon Kim, Jinkyoo Park, and Guojie Song. ReEvo: Large language models as hyper-heuristics with reflective evolution. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/4ced59d480e07d290b6f29fc8798f195-Paper-Conference.pdf.

[28] Kalyanmoy Deb, Amrit Pratap, Sameer Agarwal, and T. Meyarivan. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2):182–197, 2002. doi: 10.1109/4235.996017. URL https://ieeexplore.ieee.org/document/996017.

[29] Justin K. Pugh, Lisa B. Soros, and Kenneth O. Stanley. Quality diversity: A new frontier for evolutionary computation. *Frontiers in Robotics and AI*, 3:40, 2016. doi: 10.3389/frobt.2016.00040. URL https://www.frontiersin.org/journals/robotics-and-ai/articles/10.3389/frobt.2016.00040/full.

[30] Ewa Deelman, Karan Vahi, Gideon Juve, Mats Rynge, Scott Callaghan, Philip J. Maechling, Rajiv Mayani, Weiwei Chen, Rafael Ferreira da Silva, Miron Livny, and Kent Wenger. Pegasus, a workflow management system for science automation. *Future Generation Computer Systems*, 46:17–35, 2015. doi: 10.1016/j.future.2014.10.008. URL https://www.sciencedirect.com/science/article/pii/S0167739X14002015.

[31] Geir Kjetil Sandve, Anton Nekrutenko, James Taylor, and Eivind Hovig. Ten simple rules for reproducible computational research. *PLOS Computational Biology*, 9(10):e1003285, 2013. doi: 10.1371/journal.pcbi.1003285. URL https://journals.plos.org/ploscompbiol/article?id=10.1371/journal.pcbi.1003285.

[32] Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Larivière, Alina Beygelzimer, Florence d'Alché Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program). *Journal of Machine Learning Research*, 22(164):1–20, 2021. URL https://jmlr.org/papers/v22/20-303.html.

[33] Brian A. Nosek, Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606, 2018. doi: 10.1073/pnas.1708274114. URL https://doi.org/10.1073/pnas.1708274114.

[34] Izumi Takahara and Teruyasu Mizoguchi. Toward auditable AI scientists: A hypothesis evolution protocol for LLM agents, 2026. URL https://arxiv.org/abs/2607.09195. arXiv:2607.09195.

[35] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent LLM systems fail? In *Advances in Neural Information Processing Systems*, 2025. URL https://arxiv.org/abs/2503.13657.

[36] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with

MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://arxiv.org/abs/2306.05685.

[37] Shirley Gregor and Alan R. Hevner. Positioning and presenting design science research for maximum impact. *MIS Quarterly*, 37(2):337–355, 2013. doi: 10.25300/MISQ/2013/37.2.01. URL https://aisel.aisnet.org/misq/vol37/iss2/3/.

A Operational Details of Evolutionary Selection

The implementation distinguishes scientific assessment from deterministic population management. Candidate schemas and scoring definitions reside in `researchos/ideation/models.py`, population and portfolio rules in `population.py`, and generation scheduling and persistence in `evolution_controller.py` within the same package. The three formal score dimensions in Equation 1 are continuous values on the interval $[1, 5]$. Evidence calibration, validation feasibility, publication-profile fit, and scientific upside remain separate diagnostics or qualitative recommendations.

Candidate families use a lightweight structural similarity measure. Let $T_k(c)$ be the set of normalized tokens in field k of candidate c, with k ranging over its problem, mechanism, and contribution fields. The implementation averages token-set overlap across these fields,

$$s(a, b) = \frac{1}{3} \sum_{k \in \mathcal{K}} \frac{|T_k(a) \cap T_k(b)|}{|T_k(a) \cup T_k(b)|}, \qquad \mathcal{K} = \{\text{problem}, \text{mechanism}, \text{contribution}\}. \tag{3}$$

The overlap is defined as one when both token sets are empty and zero when exactly one is empty. Complete-link grouping requires the candidate to meet the configured similarity condition with the members of its assigned family. This deterministic similarity measure supports family grouping. Scientific overlap and contribution are assessed through the candidate evaluation and subsequent contribution review.

Survival first considers scored candidates that satisfy the structural contract and whose recorded field changes have not been classified as cosmetic or regressive. It computes Pareto layers using the three quality objectives. Within a layer, the implementation uses a lexicographic priority based on mean core score, the recorded child gain relative to its parent scores where available, and minimum structural distance to already selected candidates, followed by the explicit wildcard and high-upside flags and candidate-identity tie-breaking. A first pass applies a soft limit of two candidates per family. A backfill pass can relax this limit to reach the configured population size using existing candidates.

At most one explicitly recommended wildcard can be retained when ordinary selection would omit it. This is recorded as a qualitative exception. Structurally valid candidates without scores can remain as visibly unranked alternatives when capacity permits. Portfolio selection subsequently ranks available candidates and prioritizes different families for the short researcher-facing presentation, with its own handling of a high-upside alternative. These rules implement Pareto-based, family-aware survival and a separate portfolio policy. The complete selection rule is specified by the quality ordering, family coverage, and recorded exceptions described above.

Mutation plans identify the bottleneck, permitted changes, and fields to preserve. Approved crossover plans include compatibility decisions and gene-donor assignments. The controller retains seed records, candidate versions, field changes, scores, family assignments, and selection reasons. Accepted offspring trigger a common reassessment of the parent–offspring union. Search attempts without admitted offspring reuse the actual existing assessments. A standard cycle performs one complete update from its initial population, and additional researcher-authorized generations extend the retained state.

B Research and Execution Interfaces

The accepted plan comprises the research blueprint, claim registry, hypotheses, and experimental plan. Their identities and fingerprints are carried into the execution contract, with the stage handoffs summarized in Table 2. An external executor returns six required records. The research report describes the performed work; the result pack contains structured outcomes; the status file records completion; the run manifest identifies raw records and generated artifacts; writing-handoff facts identify observations available for reporting; and the validation receipt binds the checked files. ResearchOS verifies study identity, return status, evidence references, permissions, and file hashes before admitting the package. It then produces `drafts/t5_t8_handoff.json`, `drafts/experiment_evidence_pack.json`, and `drafts/result_to_claim.json` for downstream writing and claim checks.

C Operation and Stage Mapping

In the implementation, T1 establishes the project; T2–T3 retrieve and read literature; T3.5 synthesizes research space; and optional T3.6 prepares a survey. T4 evolves research directions, T4.5 reviews and formalizes the selected study, T5 compiles external execution, T8 develops the manuscript, and T9 assembles the final deliverables. These identifiers provide the scheduling and recovery interfaces for the three compilations.

Listing 2 shows focused T4/T8 calls and a bounded skill invocation. T4 requires research-space artifacts, while T8 requires admitted results. The `resume--from-taskT4` command reopens the direction decision and checks dependent work; ordinary `resume` continues the recorded state. Both use the same persistent workspace.

```
WS=./workspace/project-a
python -m researchos.cli run-task T4 --workspace "$WS"
python -m researchos.cli run-task T8 --workspace "$WS"
python -m researchos.cli run-skill domain-synthesis-studio \
  "Synthesize the available literature." \
  --workspace "$WS" --session-id field-review
python -m researchos.cli resume \
  --workspace "$WS" --from-task T4
```

Listing 2. Selected task, skill, and scientific re-entry commands.